Student Grade Calculator

(*A Comprehensive Software Engineering Project*

Demonstrating OOP, SOLID Principles, and Design Patterns)

Author	Syed Nokhaiz Al Hassan
Project	C++17, Student Grade Calculator

Contents

1. Executive Summary

This project implements a comprehensive Student Grade Calculator system in C++ for the University of Gujrat. It demonstrates professional software engineering practices including Object-Oriented Programming, SOLID principles, and industry-standard design patterns.

Key Achievements:

- Implemented 6 core classes with proper encapsulation and inheritance
- Applied 5 design patterns: Singleton, Strategy, Factory, Observer, and Facade
- Followed all 5 SOLID principles throughout the codebase
- Support for dynamic course enrollment and flexible grading strategies
- Robust file I/O with backup and restore functionality
- Menu-driven console interface with comprehensive user feedback
- Standalone UOG CGPA Calculator for instant GPA/CGPA computation

2. Project Overview

Purpose:

The Student Grade Calculator is designed to manage student information, track course grades, calculate GPA and CGPA values, and generate comprehensive academic reports.

Core Features:

- Student Management: Add, update, delete, and view student information
- Course Management: Enroll students in dynamic courses with flexible limits
- Grade Calculation: Automatic GPA/CGPA using UOG grading scale
- Assessment Tracking: Support for Assignments, Mid-term, Final, and Project work
- Report Generation: Academic transcripts, grade reports, and progress tracking
- Data Persistence: Save/load with backup functionality
- Grade Notifications: Alert system for low and high grades
- UOG CGPA Calculator: Standalone tool to compute GPA/CGPA from letter grades

3. University of Gujrat Grading Scale

Marks (%)	Grade	GPA	Category
84.5 and above	A+	4.00	Exceptional
79.5 - 84.4	A	3.70	Outstanding
74.5 - 79.4	B+	3.40	Excellent
69.5 - 74.4	B	3.00	Very Good
64.5 - 69.4	B-	2.50	Good
59.5 - 64.4	C+	2.00	Average
54.5 - 59.4	C	1.50	Satisfactory
49.5 - 54.4	D	1.00	Pass
Below 49.5	F	0.00	Fail

Special Grades:

W = Withdrawal | I = Incomplete | SA = Short Attendance

4. Architecture and Design

System Architecture:

The system follows a layered architecture with clear separation of concerns:
• Presentation Layer: Menu-driven console interface (main.cpp)
• Business Logic Layer: Course, Student, and GradeCalculator classes
• Data Access Layer: FileManager for persistence
• Notification Layer: GradeObserver for alerts
• Reporting Layer: ReportGenerator for various report types

Design Principles:

- Separation of Concerns: Each class has a single, well-defined responsibility
- Encapsulation: Private data members with public getters/setters
- Inheritance: Proper class hierarchy for code reuse
- Polymorphism: Virtual functions for flexible behavior
- Composition: Student contains multiple Course objects

5. OOP Concepts Implemented

Classes and Objects:
Course, Student, GradeCalculator, ReportGenerator, GradeObserver, FileManager

Encapsulation:
Private data members with public accessor methods (getters/setters)

Inheritance:
Report hierarchy (TranscriptReport, SemesterGradeReport, etc.) from Report base class

Polymorphism:
Virtual functions in Observer and Report classes

Abstraction:
Abstract Report class defines interface for all report types

Constructors:
Multiple constructors with default values

Destructors:
Proper cleanup in GradeCalculator and GradeObserver

Operator Overloading:
Can be extended (e.g., operator< for sorting)

6. Design Patterns Used

Singleton Pattern (GradeCalculator)

- Ensures only one instance of GradeCalculator exists throughout the application
- Private constructor prevents external instantiation
- Static getInstance() method returns the single instance
- Benefit: Consistent grading calculations across the system

Strategy Pattern

- Encapsulates different grading algorithms
- GradingStrategy base class with WeightedAverageStrategy and AdjustedGradingStrategy
- Allows switching between strategies at runtime
- Benefit: Easy to add new grading methods without modifying existing code

Factory Pattern (ReportGenerator)

- Creates different types of reports without exposing creation logic
- ReportFactory::createReport() method based on ReportType
- Centralized report creation
- Benefit: Easy to add new report types

Observer Pattern (GradeObserver)

- Implements Subject-Observer relationship for grade notifications
- AlertNotification and PerformanceLogger are concrete observers
- GradeChangeNotifier manages subscriptions
- Benefit: Loose coupling between grade changes and notifications

7. SOLID Principles

Single Responsibility Principle

Each class has one reason to change. Course handles marks, Student manages enrollment, GradeCalculator handles calculations.

Open/Closed Principle

Classes are open for extension (new report types, new grading strategies) but closed for modification.

Liskov Substitution Principle

Derived Report classes can replace the base Report class without breaking functionality.

Interface Segregation Principle

Classes implement only the methods they need. Observer interface segregated from Subject.

Dependency Inversion Principle

Depend on abstractions (Report, GradingStrategy) rather than concrete classes.

8. Class Structure and Relationships

Course Class

- Attributes: courseCode, courseName, creditHours, assignmentMarks, midtermMarks, finalMarks, projectMarks
- Key Methods: calculatePercentage(), getGrade(), getGradePoint()
- Responsibility: Manage individual course information and grade calculation
- Relationships: Used by Student (composition)

Student Class

- Attributes: rollNo, name, email, courses[], currentCGPA, totalCreditHoursPassed
- Key Methods: addCourse(), calculateSemesterGPA(), calculateCGPA()
- Responsibility: Manage student information and aggregate course data
- Relationships: Aggregates Course objects

GradeCalculator (Singleton)

- Attributes: currentStrategy (Strategy pattern)
- Key Methods: getInstance(), calculatePercentage(), percentageToGrade()
- Responsibility: Centralized grade calculation using UOG scale
- Relationships: Uses GradingStrategy (Strategy pattern)

9. Method Walkthroughs

Course::calculatePercentage() Method

This method calculates the overall course percentage using the weighted average strategy.
Implementation:
1. Calls GradeCalculator::getInstance() to get singleton instance
2. Uses current strategy to calculate weighted average
3. Formula: (Assignment×20%) + (Midterm×20%) + (Final×40%) + (Project×20%)
4. Returns percentage value (0-100)

Student::calculateSemesterGPA() Method

Calculates weighted GPA for current semester.
Implementation:
1. Iterates through all enrolled courses
2. For each course: totalGradePoints += (courseGPA × creditHours)
3. Divides total grade points by total credit hours
4. Returns semester GPA (0-4.0)

ReportFactory::createReport() Method

Factory method for creating different report types.
Implementation:
1. Accepts ReportType enum parameter
2. Uses switch statement to determine report type
3. Creates and returns appropriate Report subclass instance
4. Caller is responsible for deleting pointer

10. Assessment Calculation Example

Complete Calculation Walkthrough:

Assessment Component	Marks	Weight	Contribution
Assignment/Quiz	85	20%	17.0
Midterm Exam	78	20%	15.6
Final Exam	92	40%	36.8
Project Work	88	20%	17.6
		TOTAL	87.0%

Grade Conversion:

87% → Grade: A (GPA: 3.70) → Category: Outstanding

CGPA Calculation with Previous Semester:

Semester	GPA	Credit Hours	Grade Points
Previous	3.50	42	147.0
Current	3.70	36	133.2
Cumulative	-	78	280.2 ÷ 78 = 3.59

11. Compilation and Execution

System Requirements:

- C++17 compatible compiler (GCC 7+, Clang 5+, MSVC 2017+)
- Standard C++ library with filesystem support
- Minimum 50 MB RAM
- Windows, Linux, or macOS operating system

Step-by-Step: How to Run the Project

1. 1. Open a terminal (Command Prompt on Windows, Terminal on Linux/Mac) in the project folder.
2. 2. Compile the project using the following command:

- g++ -std=c++17 -Wall -I. src/main.cpp src/Course.cpp src/Student.cpp src/GradeCalculator.cpp src/ReportGenerator.cpp src/GradeObserver.cpp src/FileManager.cpp -o bin/StudentGradeCalculator
- Alternative (if Makefile is present): make

3. 5. After successful compilation, run the executable:

- Windows: bin\StudentGradeCalculator.exe
- Linux/Mac: ./bin/StudentGradeCalculator

4. 8. The main menu will appear. Use number keys to navigate and press Enter to confirm.

12. Usage Guide

Main Menu Options:

1. Student Management: Add, update, delete, view students

2. Course Management: Manage courses and enter assessment marks

3. Generate Reports: Create academic reports and transcripts

4. Settings: Change grading strategy and backup data

5. UOG CGPA Calculator: Standalone GPA/CGPA calculator (no student record needed)

6. Save & Exit: Save changes and exit application

0. Exit Without Saving

Typical Workflow:

5. 1. Add new student with basic information
6. 2. Enter optional CGPA and previous credit hours
7. 3. Add courses (up to 10 per student)
8. 4. Enter marks for each assessment component
9. 5. Generate various reports
10. 6. Save all data to file

13. Data Persistence

File Format:

Student data is stored in CSV format with pipe (|) delimiters:

RollNo|Name|Email|Semester|CGPA|CreditHours|NumCourses|CourseCode:CourseName:Credits:Assignment:Midterm:Final:Project;...

Storage Location:

Primary: data/students.csv

Backup: data/students.csv.backup

Backup & Restore:

Automatic backup creation from Settings menu

Restore functionality to recover from backup

14. Error Handling

- Input Validation: Marks must be 0-100, credit hours 1-5, email format validation
- File I/O: Graceful handling of missing files, directory creation
- Duplicate Prevention: Duplicate course codes are not allowed per student
- Limits: Maximum 10 courses per student, 500 students max
- Memory Management: Proper cleanup in destructors, no memory leaks

15. Conclusion and Future Enhancements

This Student Grade Calculator project successfully demonstrates professional software engineering practices including OOP, SOLID principles, and design patterns. The system is extensible and maintainable, providing a solid foundation for future enhancements.

Possible Future Enhancements:

- GUI using Qt or wxWidgets library
- Database integration (SQLite or MySQL)
- Web interface using modern web frameworks
- Mobile application for student access
- Automated email notifications
- Grade prediction using machine learning
- Advanced analytics and performance charts
- Multi-user authentication system

16. UOG CGPA Calculator

The UOG CGPA Calculator is a standalone, interactive feature accessible directly from the main menu (Option 5). It allows any student to calculate their Semester GPA and Cumulative CGPA by entering letter grades — without needing to create a student record first.

How to Use:

11. Select option 5 "UOG CGPA Calculator" from the main menu.
12. The system displays the full UOG grade-to-grade-point reference table.
13. Enter your current CGPA (optional — press Enter to skip if you are in your 1st semester).
14. If a CGPA was entered, also enter the total credit hours completed so far.
15. Enter the number of courses for this semester.
16. For each course: enter an optional course name, the letter grade (e.g. A+, B, C+), and credit hours (1-5). The system immediately shows the grade points and contribution.
17. After all courses, the system displays a full summary table and calculates:

- - Total grade points (sum of grade × credits for each course)
- - Semester GPA
- - Updated CGPA (if previous CGPA was provided)

Calculation Formulas:

Semester GPA = Sum(Grade Points × Credit Hours) / Total Credit Hours

Updated CGPA = (Previous CGPA × Previous Credits + Semester Grade Points) / (Previous Credits + Semester Credits)

Example:

Course	Grade	Grade Points	Credits	Contribution
Data Structures	A+	4.00	3	12.00
OOP in C++	A	3.70	3	11.10
Digital Logic	B+	3.40	3	10.20
Calculus	B	3.00	3	9.00
English	A+	4.00	2	8.00
TOTALS			14	50.30

Semester GPA = 50.30 / 14 = 3.59

If previous CGPA = 3.40 with 42 credits:

Updated CGPA = (3.40 × 42 + 50.30) / (42 + 14) = 192.50 / 56 = 3.44

Chapter: File-by-File Code Explanation

This chapter provides a thorough explanation of every header (.h) and source (.cpp) file in the Student Grade Calculator project. Each file description covers: purpose, class/struct declarations, private members, public interface, and how it connects to the rest of the system.

1. include/Constants.h

Constants.h is the single source of truth for all magic numbers in the project. It defines no classes only global const values grouped into four logical sections. Every other file includes this header directly or indirectly.

1.1 Grading Scale Thresholds

Nine float constants define the minimum percentage required to earn each letter grade on the University of Gujrat scale:

```
const float GRADE_AP_MIN = 84.5f;  // A+  (Exceptional)
const float GRADE_A_MIN  = 79.5f;  // A   (Outstanding)
const float GRADE_BP_MIN = 74.5f;  // B+  (Excellent)
const float GRADE_B_MIN  = 69.5f;  // B   (Very Good)
const float GRADE_BM_MIN = 64.5f;  // B-  (Good)
const float GRADE_CP_MIN = 59.5f;  // C+  (Average)
const float GRADE_C_MIN  = 54.5f;  // C   (Satisfactory)
const float GRADE_D_MIN  = 49.5f;  // D   (Pass)
const float GRADE_F_MIN  =  0.0f;  // F   (Fail)
```

1.2 GPA Point Values

Paired GPA values for each grade:

```
GPA_AP=4.00  GPA_A=3.70  GPA_BP=3.40  GPA_B=3.00  GPA_BM=2.50
GPA_CP=2.00  GPA_C=1.50  GPA_D=1.00   GPA_F=0.00
```

1.3 Assessment Component Weights

```
ASSIGNMENT_WEIGHT = 0.20f  (20%)
MIDTERM_WEIGHT    = 0.20f  (20%)
FINAL_WEIGHT      = 0.40f  (40%)
PROJECT_WEIGHT    = 0.20f  (20%)
```

1.4 Validation Limits & Notification Thresholds

- MAX_STUDENTS = 500, MAX_COURSES_PER_STUDENT = 10
- Marks range: 0 – 100; Credit hours range: 1 – 5
- LOW_GRADE_THRESHOLD = 2.0 → triggers warning alert
- HIGH_GRADE_THRESHOLD = 3.7 → triggers congratulation message
- DATA_FILE = "data/students.csv", BACKUP_FILE = "data/backup.csv"

2. include/Course.h

Course.h declares the Course class — the fundamental data unit of the system. A single Course object represents one academic subject a student is enrolled in for the semester. It stores all four assessment mark components and exposes methods to compute the weighted grade. The class depends on GradeCalculator (via delegation) for all calculation logic, keeping it slim and focused.

2.1 Private Data Members

- **courseCode (string):** Unique course identifier, e.g. "CS-201". Used to prevent duplicate enrolment.
- **courseName (string):** Full human-readable course name.
- **creditHours (int):** Credit weight used in GPA formula; must be between 1 and 5.
- **assignmentMarks (float):** Raw score out of 100. Weighted at 20% in the final percentage.
- **midtermMarks (float):** Raw score out of 100. Weighted at 20%.
- **finalMarks (float):** Raw score out of 100. Weighted at 40% (heaviest component).
- **projectMarks (float):** Raw score out of 100. Weighted at 20%.
- **isCompleted (bool):** Signals that the course is done; used by Student to filter credit-hour totals.

2.2 Constructors

- Course() — Default: sets all strings to empty, numeric fields to 0, isCompleted to false.
- Course(code, name, credits) — Parameterised: initialises identity fields; all marks start at 0.

2.3 Key Public Methods

Setters (Encapsulation)

All mark setters first call isValidMarks(marks) to enforce the 0–100 range before assignment. setCreditHours() validates against MIN_CREDIT_HOURS (1) and MAX_CREDIT_HOURS (5).

Core Calculation Methods

- calculateTotalMarks() / calculatePercentage() → Calls GradeCalculator::getInstance()->calculatePercentage() with the four stored marks. This delegates the formula to whichever grading strategy is active (Standard or Adjusted).
- getGrade() → Calls GradeCalculator::percentageToGrade(total). Returns "A+", "A", "B+", etc.
- getGradePoint() → Calls GradeCalculator::percentageToGPA(total). Returns 4.00, 3.70, etc.
- getCategory() → Calls GradeCalculator::percentageToCategory(total). Returns "Exceptional", "Pass", etc.

Display & Validation

- displayCourseInfo() → Prints course code, name, credit hours, and completion status.

- displayGradeInfo() → Prints all four raw marks plus computed %, grade, GPA, and category.
- isAllMarksEntered() → Returns true only when all four mark fields are > 0; used by UI to warn about incomplete entries.

3. src/Course.cpp

Implements every method declared in Course.h. Includes Course.h and GradeCalculator.h (for delegation), plus <iostream> and <iomanip> for formatted console output. The key design decision is that Course itself stores NO calculation logic — it delegates every grade computation to GradeCalculator::getInstance().

3.1 Setter Validation Pattern

Every numeric setter follows the same guard pattern:

```
void Course::setAssignmentMarks(float marks) {
    if (isValidMarks(marks))       // 0 <= marks <= 100
        assignmentMarks = marks;   // only assign if valid
}
```

This enforces data integrity at the boundary — invalid values are silently ignored, which is appropriate for a menu-driven system where the UI already validates user input.

3.2 Delegation to GradeCalculator

The three core grade methods all follow the same delegation pattern:

```
float Course::calculateTotalMarks() const {
    GradeCalculator* calc = GradeCalculator::getInstance();
    return calc->calculatePercentage(assignmentMarks, midtermMarks,
finalMarks, projectMarks);
}

std::string Course::getGrade() const {
    GradeCalculator* calc = GradeCalculator::getInstance();
    return calc->percentageToGrade(calculateTotalMarks());
}
```

This approach means changing the grading algorithm (e.g. switching to AdjustedGradingStrategy) automatically affects every Course without modifying Course.cpp.

3.3 Display Methods

displayCourseInfo() prints a 70-character dashed separator followed by the four identity fields. displayGradeInfo() prints all raw marks with std::fixed/setprecision(2) formatting, plus the computed percentage, grade letter, GPA value, and performance category.

4. include/Student.h

Student.h declares the Student class — the top-level aggregate entity of the system. A Student owns a vector<Course> (up to 10 courses), stores their academic identity and previous CGPA context, and provides the GPA/CGPA calculation engine plus comprehensive display/reporting methods. This is the class that most of the rest of the system interacts with.

4.1 Private Data Members

- **rollNo (string):** Unique student identifier (e.g. "21-CS-01"). Used for look-ups and file records.
- **name (string):** Full student name displayed on all reports.
- **email (string):** Set only after regex validation passes. Format: user@domain.ext.
- **courses (vector<Course>):** Dynamic list of enrolled courses; capacity capped at MAX_COURSES_PER_STUDENT (10).
- **currentCGPA (float):** Accumulated CGPA earned in previous semesters (0.0 for first semester).
- **totalCreditHoursPassed (int):** Credit hours completed before the current semester; used in CGPA blending.
- **semesterNumber (int):** Current semester number (1-based).

4.2 Course Management Methods

- addCourse(course) → Double-guarded: checks (1) courses.size() < MAX_COURSES_PER_STUDENT, and (2) !isCourseCodeExists(code) to prevent duplicates. Prints success or error message.
- removeCourse(index) → Bounds-checked erase using vector::erase.
- getCourse(index) — two overloads → mutable version (for mark entry) and const version (for report generation).
- isCourseCodeExists(code) → Linear scan over all enrolled courses; prevents two CS-201 entries.
- findCourseByCode(code) → Returns index or -1; used by UI to locate courses by code string.

4.3 GPA / CGPA Calculation

- calculateSemesterGPA() → Σ(course.getGradePoint() × course.getCreditHours()) ÷ Σ(creditHours) for all enrolled courses.
- calculateTotalGradePoints() → Raw sum of grade_point × credit_hours; used as numerator in CGPA blend.
- calculateTotalCreditHours() → Sums credits only for courses where isCompleted == true.
- calculateCGPA() → Full formula: (currentCGPA × totalCreditHoursPassed + calculateTotalGradePoints()) ÷ (totalCreditHoursPassed + calculateTotalCreditHours()). Falls back to semester GPA for first-semester students.

4.4 Display / Report Methods

- displayStudentInfo() → Roll number, name, email, semester number, and course count.
- displayAllCourses() → 100-character wide formatted table with columns: No. | Code | Name | Credit | Percentage | Grade | GPA | Category.
- displayTranscript() → Calls displayStudentInfo + displayAllCourses + GPA summary block.
- displaySemesterSummary() → Semester-level header + course count + semester GPA + course table.
- displayDetailedReport() → Full transcript + per-course displayCourseInfo() and displayGradeInfo() for every enrolled course.

4.5 Validation

- isValidEmail(email) → Uses std::regex with pattern: [a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}. Returns true only on a full match.

5. src/Student.cpp

Implements all Student methods. Includes <regex> for email validation, <algorithm> for course searching, and <iomanip> for formatted table output. The GPA and CGPA calculation are pure C++ arithmetic — no external dependencies.

5.1 CGPA Calculation Logic

The calculateCGPA method implements the weighted-average blending formula:

```
float Student::calculateCGPA() const {
    float totalCredits = totalCreditHoursPassed +
calculateTotalCreditHours();
    if (totalCredits == 0)
        return calculateSemesterGPA();  // first-semester fallback
    float totalGradePoints = (currentCGPA * totalCreditHoursPassed)
                           + calculateTotalGradePoints();
    return totalGradePoints / totalCredits;
}
```

5.2 Course Addition Guard

```
void Student::addCourse(const Course& course) {
    if (courses.size() < MAX_COURSES_PER_STUDENT) {
        if (!isCourseCodeExists(course.getCourseCode())) {
            courses.push_back(course);
        } else { /* "Course code already exists" */ }
    } else { /* "Maximum courses limit reached" */ }
}
```

5.3 Email Validation with std::regex

```
bool Student::isValidEmail(const std::string& email) const {
    std::regex pattern(R"([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-
Z]{2,})");
    return std::regex_match(email, pattern);
}
```

5.4 Formatted Table Output

displayAllCourses() uses std::setw with specific column widths to produce an aligned 100-character wide table. It uses std::left for text fields and std::fixed + std::setprecision(2) for floating-point values.

6. include/GradeCalculator.h

GradeCalculator.h is the most pattern-rich header in the project. It declares three classes implementing two design patterns simultaneously: the Strategy pattern (swappable grading algorithms) and the Singleton pattern (one shared calculator object).

6.1 GradingStrategy — Abstract Interface (Strategy Pattern)

An abstract base class that defines the contract for any grading algorithm:

```
class GradingStrategy {
public:
    virtual ~GradingStrategy() = default;
    virtual float calculatePercentage(float assignment, float
midterm,
                                      float final, float project)
const = 0;
    virtual std::string getStrategyName() const = 0;
};
```

The pure virtual destructor ensures safe polymorphic cleanup when strategies are swapped at runtime.

6.2 WeightedAverageStrategy — Concrete Strategy #1

Implements the standard University formula: assignment×0.20 + midterm×0.20 + final×0.40 + project×0.20. This is the DEFAULT strategy, selected automatically when GradeCalculator is first constructed. getStrategyName() returns "Weighted Average (Standard)".

6.3 AdjustedGradingStrategy — Concrete Strategy #2

Calculates the same weighted base percentage, then applies a +5 curve if the final exam score exceeds 80, capped at 100% maximum via std::min(base + 5.0f, 100.0f). getStrategyName() returns "Adjusted Grading (With Curve)". Swappable at runtime via GradeCalculator::setGradingStrategy().

6.4 GradeCalculator — Singleton Class

Implements the Singleton pattern to ensure exactly one calculator exists throughout the application:

- Private constructor: creates a WeightedAverageStrategy on the heap; assigned to currentStrategy.
- static GradeCalculator* instance = nullptr; — starts null; allocated lazily on first getInstance() call.
- getInstance() → if(instance==nullptr) instance=new GradeCalculator(); return instance;
- Deleted copy constructor and assignment operator prevent accidental copies.
- setGradingStrategy(strategy) → deletes the old strategy, assigns the new one (runtime algorithm swap).

- Grade conversion methods: percentageToGrade(), percentageToGPA(), percentageToCategory() — all use if-else chains referencing Constants.h thresholds.

7. src/GradeCalculator.cpp

Provides the full implementation of both strategy classes and the GradeCalculator singleton. Uses <cmath> for std::min (grade curve cap) and <iomanip> for the displayGradingTable output.

7.1 Strategy Implementations

```
// Standard — pure weighted average
float WeightedAverageStrategy::calculatePercentage(
        float a, float m, float f, float p) const {
    return a*ASSIGNMENT_WEIGHT + m*MIDTERM_WEIGHT
         + f*FINAL_WEIGHT      + p*PROJECT_WEIGHT;
}

// Adjusted — adds 5-point curve when final exam > 80
float AdjustedGradingStrategy::calculatePercentage(
        float a, float m, float f, float p) const {
    float base = a*0.20 + m*0.20 + f*0.40 + p*0.20;
    if (f > 80.0f) return std::min(base + 5.0f, 100.0f);
    return base;
}
```

7.2 Singleton Lifecycle

- Static initialisation: GradeCalculator* GradeCalculator::instance = nullptr; — defined once at file scope.
- Private constructor allocates new WeightedAverageStrategy() and stores it in currentStrategy.
- getInstance() performs lazy initialisation — object created on first access, not at program start.
- Destructor deletes currentStrategy to release heap memory, preventing leaks on program exit.
- setGradingStrategy() first deletes the old strategy, then assigns the new pointer — no double-free risk.

7.3 Grade Conversion Chain

Each of the three conversion methods (percentageToGrade, percentageToGPA, percentageToCategory) uses an identical if-else waterfall that checks from highest threshold to lowest, referencing constants from Constants.h:

```
std::string GradeCalculator::percentageToGrade(float pct) const {
    if (pct >= GRADE_AP_MIN) return "A+";
    if (pct >= GRADE_A_MIN)  return "A";
    if (pct >= GRADE_BP_MIN) return "B+";
    // ... continues down to D
    return "F";
}
```

8. include/ReportGenerator.h

Declares a family of report-generating classes organised around two design patterns: a polymorphic Report hierarchy (Template Method / Strategy feel) created through a Factory, and a Facade (ReportManager) that simplifies calling code.

8.1 Report — Abstract Base Class

- Pure virtual generate(const Student& student) → each concrete type implements its own output.
- Pure virtual getReportType() → returns the report name as a string.
- Virtual destructor allows safe polymorphic deletion via base pointer.

8.2 Four Concrete Report Classes

- TranscriptReport → Full academic history: student info + all courses + GPA + CGPA.
- SemesterGradeReport → Current-semester grade table with semester GPA footer.
- CGPAProgressReport → Previous CGPA vs updated CGPA with IMPROVED/DECLINED delta.
- DetailedAnalysisReport → Per-course breakdown: raw marks for each assessment component.

8.3 ReportFactory — Factory Pattern

A static factory that decouples object creation from usage:

```
static Report* createReport(ReportType type);
// Enum: TRANSCRIPT | SEMESTER_GRADE | CGPA_PROGRESS | DETAILED_ANALYSIS
```

Callers request a report type without knowing which concrete class handles it. The factory allocates the object on the heap; the caller is responsible for deleting it (handled by ReportManager).

8.4 ReportManager — Facade Pattern

- generateAndDisplayReport(student, type) → creates report via factory, calls generate(), deletes pointer.
- generateAllReports(student) → calls generateAndDisplayReport() for all four types in sequence.
- Both methods are static — ReportManager has no state, making it inherently thread-safe.

9. src/ReportGenerator.cpp

Implements all report generate() methods plus the Factory and Facade. Each report method only calls Student const methods — no direct member access — preserving encapsulation. All output uses std::setw column alignment.

9.1 TranscriptReport::generate()

- Prints 100-char "=" banner with "ACADEMIC TRANSCRIPT — UNIVERSITY OF GUJRAT" header.
- Calls student.displayStudentInfo() for identity block.
- Calls student.displayAllCourses() for the full course table.
- Footer block: Semester GPA, Previous CGPA, Previous Credit Hours, Updated CGPA.

9.2 SemesterGradeReport::generate()

- Prints semester number and student name/roll number header.
- Manually iterates all courses to accumulate totalGradePoints and totalCredits.
- Prints one formatted row per course: Code | Name | Marks% | Grade | GPA | Category.
- Footer line: Semester GPA = totalGradePoints / totalCredits.

9.3 CGPAProgressReport::generate()

- Shows Previous CGPA, Previous Credit Hours, Semester GPA, Semester Credit Hours.
- Calculates difference = calculateCGPA() - getCurrentCGPA().
- Labels result: [IMPROVED] if positive, [DECLINED] if negative, [NO CHANGE] if zero.

9.4 DetailedAnalysisReport::generate()

- For each course: prints header [Course X/N], code, name, credit hours.
- Assessment breakdown shows raw marks with weight annotations: Assignment/100 (20%).
- Grade results: overall %, grade letter, GPA points, performance category.
- Overall footer: Semester GPA and updated CGPA.

9.5 Factory & Manager

- ReportFactory::createReport() → switch(type) allocates the matching concrete class.
- generateAndDisplayReport() → Report* r = factory; r->generate(s); delete r; — no memory leak.
- generateAllReports() → four sequential generateAndDisplayReport() calls wrapped in banners.

10. include/GradeObserver.h

Declares the complete Observer (Publish-Subscribe) notification system. Five classes implement the full pattern: abstract Observer interface, two concrete listeners, a Subject that maintains the subscriber list, and a Singleton orchestrator.

10.1 Observer — Abstract Interface

- Pure virtual update(courseName, gpa, category) → called by Subject whenever a grade changes.
- Virtual destructor ensures derived class destructors are called correctly on polymorphic delete.

10.2 AlertNotification — Concrete Observer #1

- Stores observerName ("Low Grade Alert System" by default).
- update(): if gpa < 2.0 → WARNING banner with advice; if gpa >= 3.7 → Congratulations banner.
- Silent (no output) for grades in the 2.0–3.69 normal range.

10.3 PerformanceLogger — Concrete Observer #2

- Stores observerName ("Performance Logger" by default).
- update(): always prints a [LOG] line regardless of grade level — provides a full session audit trail.

10.4 GradeSubject — Observable (Subject)

- Holds vector<Observer*> observers — the subscriber list.
- attach(observer) → appends to list if not nullptr.
- detach(observer) → linear search, erase, delete the pointer.
- notify(courseName, gpa, category) → calls update() on every observer in the list.
- Destructor iterates and deletes every remaining observer to prevent leaks.

10.5 GradeChangeNotifier — Singleton Orchestrator

- Singleton: private constructor, static instance*, deleted copy/assignment.
- Owns a GradeSubject internally — callers never interact with GradeSubject directly.
- subscribeLowGradeAlert() → new AlertNotification → attached to subject.
- subscribePerformanceLogger() → new PerformanceLogger → attached to subject.
- checkAndNotify(course) → reads course.getGradePoint(); fires only when threshold crossed (< 2.0 or >= 3.7).

11. src/GradeObserver.cpp

Implements all five Observer-pattern classes. GradeSubject is the memory owner of all observers (deletes them in its destructor). GradeChangeNotifier acts as the bridge between the mark-entry UI and the notification infrastructure.

11.1 Observer Implementations

- AlertNotification::update() → Threshold check against LOW_GRADE_THRESHOLD and HIGH_GRADE_THRESHOLD from Constants.h; prints star-bordered alert or celebration block.
- PerformanceLogger::update() → Simple "[LOG] observer — grade update" line for every notification regardless of level.

11.2 GradeSubject Subject Management

```
void GradeSubject::attach(Observer* o)  { if(o)
observers.push_back(o); }
void GradeSubject::detach(Observer* o)  {
    for(auto it=observers.begin(); it!=observers.end(); ++it)
        if(*it==o) { observers.erase(it); delete o; break; }
}
void GradeSubject::notify(name, gpa, cat) {
    for(auto obs : observers) obs->update(name, gpa, cat);
}
```

11.3 GradeChangeNotifier — Integration Point

Called from main.cpp at startup to wire up notifications:

```
GradeChangeNotifier::getInstance()->subscribeLowGradeAlert();
GradeChangeNotifier::getInstance()->subscribePerformanceLogger();
```

Then called after every mark update:

```
GradeChangeNotifier::getInstance()->checkAndNotify(course);
// → reads grade point, fires notify only when threshold crossed
```

12. include/FileManager.h

Declares the FileManager class — the entire Data Access Layer of the project. Responsible for serialising Student objects to/from a pipe-delimited CSV file, creating and restoring backups, and managing the data/ directory. It is the only class in the project that performs file I/O.

12.1 Private Helper Methods

- studentToCSV(student) → Converts a complete Student object (with all courses) into one CSV line.
- csvToStudent(line) → Parses one CSV line back into a fully populated Student object.
- directoryExists(path) → Uses std::filesystem::exists + is_directory.
- createDirectory(path) → Uses std::filesystem::create_directory.

12.2 Public File Operations

- FileManager(path) → Constructor; immediately creates data/ directory if it does not exist.
- saveStudents(vector<Student>) → Overwrites file with ALL students (full serialisation).
- loadStudents() → Reads every line, deserialises each into a Student, returns the vector.
- saveStudent(student, append) → Single-student save with append or overwrite flag.
- deleteStudentRecord(rollNo) → Load all → filter out matching roll → saveStudents (atomic rewrite).

12.3 File Utilities

- fileExists() → ifstream probe: file.good() returns true if the file can be opened.
- createBackup() → fs::copy(filePath, filePath+".backup", overwrite_existing); wrapped in try/catch.
- restoreFromBackup() → Reverse copy; returns false if no backup file found.
- clearData() → Opens file with ios::trunc to truncate contents to zero bytes.
- getRecordCount() → Counts non-empty lines by iterating with getline.

12.4 CSV Record Format

Each student is stored as a single line in this format:

```
rollNo|name|email|semesterNo|cgpa|creditHours|courseCount|
courseCode:courseName:credits:assignment:midterm:final:project;
courseCode:courseName:credits:assignment:midterm:final:project;...
```

Pipe (|) separates student-level fields. Colon (:) separates course sub-fields. Semicolon (;) delimits individual course entries within the courses block.

13. src/FileManager.cpp

Implements all FileManager methods. Uses <fstream> for I/O, <sstream> for string parsing, <filesystem> (C++17) for backup and directory operations, and <algorithm> for the remove_if delete pattern.

13.1 studentToCSV() Serialisation

Builds the CSV line using a stringstream:

```
ss << rollNo << "|" << name << "|" << email << "|"
   << sem << "|" << cgpa << "|" << credits << "|" << count << "|";
for each course:
    ss << code << ":" << name << ":" << credits << ":"
       << assignment << ":" << midterm << ":" << final << ":" <<
project << ";";
ss << "\n";
```

13.2 csvToStudent() Deserialisation (Three-Level Parse)

- Level 1 — Student fields: getline(ss, field, "|") extracts the 7 student header fields.
- Level 2 — Course blocks: getline(coursesSS, courseData, ";") splits each course entry.
- Level 3 — Course fields: getline(courseSS, field, ":") extracts the 7 per-course values.
- std::stoi / std::stof convert the numeric strings; all setters are called individually.
- student.addCourse(course) adds each reconstructed course to the student.

13.3 deleteStudentRecord() Pattern

```
vector<Student> students = loadStudents();  // read all
auto it = remove_if(students.begin(), students.end(),
    [&](const Student& s){ return s.getRollNo() == rollNo; });
students.erase(it, students.end());         // compact vector
return saveStudents(students);               // rewrite file
```

This load-filter-save approach ensures the file is always in a consistent state — there is no in-place deletion.

13.4 Backup via std::filesystem

- createBackup() → fs::copy(source, dest, overwrite_existing) — one call with error handling.
- restoreFromBackup() → Checks fs::exists(backup) before copying to avoid crashing on missing backup.
- Both operations use try/catch(std::exception) to print meaningful error messages.

14. src/main.cpp

The Presentation Layer — the only file the user interacts with directly. Contains the main() function, all menu display functions, and all CRUD operation handlers. Uses two global objects as the in-memory database and file interface, then orchestrates every other class through the menu system.

14.1 Global State

```
vector<Student> students;                          // in-memory
student database
FileManager fileManager("data/students.csv");      // shared file
handle
```

14.2 Startup Sequence in main()

- SetConsoleOutputCP(65001) → enables UTF-8 output so Unicode symbols (✓ ✗ ⚠ 🎉) render correctly on Windows.
- GradeChangeNotifier::getInstance()->subscribeLowGradeAlert() → wires up warning alerts.
- GradeChangeNotifier::getInstance()->subscribePerformanceLogger() → wires up the audit logger.
- students = fileManager.loadStudents() → loads any previously saved data into memory.
- Enters the main while(running) loop driven by a 6-option switch statement.

14.3 Main Menu Structure

- 1. Student Management → add / update / delete / list all / view details.
- 2. Course Management → add course to student / enter/update marks / remove course / view courses.
- 3. Report Generation → select student → choose specific report or generate all.
- 4. Settings → change grading strategy / view grading table / view file info.
- 5. UOG CGPA Calculator → standalone instant calculator without needing a stored record.
- 6. Save & Exit → fileManager.saveStudents(students); running=false.

14.4 Student CRUD Functions

- addStudent() → prompts for roll/name/email + optional previous CGPA and credits; creates Student object; push_back to students vector.
- updateStudent() → find by roll number in vector; re-prompts for mutable fields; writes back.
- deleteStudent() → find by roll number; erase from vector; calls fileManager.deleteStudentRecord(rollNo) for persistence.
- listStudents() → formatted table view: roll no | name | semester | courses | CGPA.
- viewStudentDetails() → calls student.displayDetailedReport() for a selected student.

14.5 Course & Mark Entry Functions

- addCourseToStudent() → select student → enter code, name, credit hours → student.addCourse().
- updateCourseMarks() → select student → select course by index → enter all 4 marks → course.setXxxMarks() → GradeChangeNotifier::checkAndNotify() to trigger alerts if threshold crossed.
- removeCourseFromStudent() → find course by code → student.removeCourse(index).
- viewCoursesOfStudent() → select student → student.displayAllCourses().

14.6 UOG CGPA Calculator (uogCGPACalculator)

A self-contained function that needs NO stored student record:

- Prompts for previous CGPA and completed credits (can be skipped for 1st semester with 0 / 0).
- Prompts for the number of courses this semester.
- For each course: user enters letter grade (e.g. "A+"); system uses GradeCalculator::getInstance() to look up the GPA value; credit hours entered; contribution = GPA × credits.
- Shows formatted summary table of all courses.
- Prints Semester GPA = Σ(GPA×credits) / Σcredits.
- Prints Updated CGPA = (prevCGPA×prevCredits + semGradePoints) / (prevCredits + semCredits).

THE END